A Static-Analysis Gate for Adaptive Retrieval: Cutting Structural Hallucinations in Repository-Level Code Completion

first last name
Delft University of Technology
Delft, Netherlands
email@tudelft.nl

Second Author
Delft University of Technology
Delft, Netherlands
email@tudelft.nl

Abstract—Retrieval-augmented generation (RAG) improves repository-level code completion, but retrieving on every request is wasteful and can even degrade output. Adaptive methods such as CARD gate retrieval on the model's token-level confidence; this signal, however, cannot account for hallucinated identifiers that do not exist in the repository. We add a lightweight, training-free static-analysis gate on top of CARD's confidence gate: when CARD decides to skip retrieval, we parse the zero-shot completion and trigger retrieval if it uses an out-of-scope (unresolvable) identifier. We evaluate the resulting cascade on four Python benchmarks spanning single-line (CrossCodeEval), multi-line function (RepoEval, CrossCodeLongEval), and multi-line block (CrossCodeLongEval) completion with three code LMs (Qwen2.5-Coder-0.5B/1.5B and CodeLlama-7B), sweeping the retrieval budget end to end. Across all twelve (evaluation dataset × model) settings the static gate cuts the rate of hallucinated identifiers (names absent from the ground truth that also resolve nowhere) by 2–8× over CARD at a matched, single-digit-percent retrieval budget, without sacrificing edit similarity. We further find that always-retrieving does not reduce invented identifiers: in our setup it increases them in every single setting, while improving accuracy only modestly — so the structural signal addresses a failure mode that neither raw retrieval nor confidence gating handles. We release code, the per-instance results, and a reproducible evaluation harness.

Index Terms—code completion, retrieval-augmented generation, adaptive retrieval, code language models, static analysis, hallucination, repository-level context

I. Introduction

Modern software rarely lives in a single file: a typical edit depends on helpers, types, and constants declared elsewhere in the project. Repository-level code completion mirrors this reality — the model completes code inside a real project rather than an isolated snippet — and is therefore the setting closest to how completion tools are actually used. Code completion thus requires assistance from beyond the current file, as utility functions, shared variables and imported types can be defined elsewhere in the project. Retrieval Augmented Generation [1] addresses this need by augmenting the model's prompt with cross-file-relevant snippets retrieved from the repository before generation. However, retrieval doesn't always help due to pipelines either retrieving on every instance unconditionally and not taking into account the local context or adaptive approaches relying too much on model's confidence. Therefore, current approaches such as CARD [2] share a common issue: token-level confidence not distinguishing a syntactically correct line from one whose identifiers do not appear in the repository, not being able to detect structural hallucinations. To address this gap, we augment current models' confidence gate with a static analysis gate, which checks whether the identifiers the completion uses are defined in the scope of the current file using the pyflakes linter. The cascade pipeline operates in 3 stages: zero-shot generation, CARD's confidence gate, and our static analysis gate. The results of our experiments suggest that token-level confidence alone is insufficient and that lightweight structural checks can help in avoiding structural hallucinations. Our main contribution consists of the static analysis gate approach which detects out-of-scope identifiers hallucinations without the need for fine tuning the model.

II. Problem Statement and Motivation

Instruction: Choose a project from the list provided to you. Clearly define the chosen task and explain why it intrigues your team (state why should people care about this topic).

Task. Given the code before and after a cursor position in one file of a repository (the in-file context X), the system must produce the missing span — a line, a fixed block, or an entire function body — optionally after retrieving cross-file context from the rest of the project. We study repository-level completion rather than isolated-snippet completion because it is the realistic deployment setting: in real projects the correct completion routinely names helpers, classes, or constants that are defined in other files, so a model restricted to the current file must either know the project from training (it usually cannot — the benchmarks are curated against that) or guess. Why care. Guessing produces a particularly damaging failure: code that looks right but calls things that do not exist, which type-checks the developer's patience rather than the program. Retrieval can fix this but is too expensive to run

on every keystroke, so the practical question — and the core of this project — is when retrieval is worth its cost, and on what evidence that decision should be made.

A. Research Questions

Instruction: Here it should be a story of RQs and how they relate and why we define them, rather than listing them. furthermore, we will have subsections for evaluation metrics, datasets (for both training testing), baselines, and configuration details. note that if dataset creation is an important step of your work, you can move it to the approach section.

Our study is organised around four questions that build on one another. We first ask whether retrieval is even worth gating: RQ1 — Is cross-file retrieval uniformly beneficial, or selective? If retrieval helped every instance, an always-retrieve policy would suffice and adaptivity would be pointless from an accuracy perspective; if it is unhelpful or harmful on a subset, a gate is needed. Once a gate is justified, we examine the state-of-the-art confidence gate: RQ2 — When CARD's confidence threshold is varied, how much accuracy does each percent of retrieval buy, and which errors still slip through when the model is confident? The threshold controls the retrieval budget (the fraction of instances on which retrieval is performed and paid for), so RQ2 asks two things: how efficiently the confidence gate converts budget into accuracy, and what kind of mistakes remain in the completions it confidently keeps. We hypothesise that confidence correlates with accuracy but is blind to structural validity, letting confident yet unresolvable completions through. This motivates our contribution: RQ3 — Does a training-free static-analysis gate suppress the hallucinated identifier that the confidence gate misses, and at what additional retrieval cost? Finally, because a gate that improves one metric by harming another is not useful, RQ4 — Does the static gate preserve accuracy and latency, and how do the effects scale with model size and task granularity? RQ1–RQ2 establish the setting and baseline, RQ3 measures the contribution, and RQ4 checks that it comes for free. The remainder of this section defines the evaluation metrics, datasets, baselines, and configuration used to answer them.

III. Background and Related Work

Instruction: To ensure a comprehensive understanding and position your work in the broader landscape context, it is essential to familiarize yourself with recent advancements in the domain. Search for recent work that addresses your specific problem. Moreover, research studies that use your proposed model/method and summarise them. Use tools like Google Scholar, Semantic Scholar, or DBLP for your search. Prioritize contributions from top software engineering conferences such as ICSE, ESEC/FSE, ASE, and MSR and well-known journals like IEEE TSE, ACM TOSEM, and Springer EMSE. Find patterns in the relevant literature for instance work on memorization, LLMs, and attacks. etc. keep it short. make sure to highlight the gaps that you fill, what you add to the existing body of knowledge

A. Large Language Models (solution)

Instruction: Describe background knowledge, e.g., what is an LLM, fine-tuning, prompt engineering, chain of thought, instruct tuning, etc (any of which you use in your work)

B. Bug Localization (problem)

Instruction: Give relevant work on the problem you are solving, for example, you are proposing a bug localization approach, or you are working on benchmarks, etc. Study the relevant work and present them here.

C. Add more sub-headings based on your work

IV. Approach

A. Design Rationale

CARD's adaptive-retrieval gate [2] predicts the zero-shot output's edit similarity from a 13-dimensional aggregation of per-token probabilities and entropies, and skips retrieval whenever this prediction exceeds a threshold T_{RAG}. The signal is cheap and empirically strong, but it is structurally superficial: token-level confidence cannot distinguish a syntactically correct line from one whose identifiers do not exist. We complement CARD with a second gate that parses the structure of the zero-shot prediction rather than the model's confidence, and triggers retrieval whenever the prediction uses an identifier that cannot be resolved in the current file's scope, catching the structural hallucination the logit signal missed.

Because their decisions are not symmetric, we compose the two gates by subordination: the static gate runs only when CARD skips it. A static fire is positive evidence of a hallucination, while the absence of a static fire is not positive evidence of correctness, only that the prediction violates none of the patterns the gate enforces. The static gate is therefore well-suited to supplement CARD's skip decisions but cannot meaningfully reject its retrieve decisions. It is also targeted at the residual failure mode CARD's signal lets through: confidently generated outputs that nonetheless contain hallucinations.

B. Pipeline

Given the in-file context $X = (X_{\text{left}}, X_{\text{right}})$ surrounding the completion hole in the repository R, the cascade produces a prediction $\hat{y}$ in three stages.

Stage 1: zero-shot generation. The code language model G produces $\hat{y}_0 = G(X_{\text{left}}, X_{\text{right}})$ together with per-token log-probabilities. We extract CARD's 13-D feature vector from the logits and compute $\hat{s}_0 = \text{Estimator}(\cdot)$, as proposed in the original paper. Concretely, the 13 features are six summary statistics (maximum, minimum, mean, standard deviation, product, and geometric mean) over the chosen-token probabilities of the generated sequence,

the same six statistics over the per-token entropies, plus the generation length — a fixed-size summary of how confident the model was, token by token, while writing $\hat{y}_0$. The estimator is a small learned regressor (gradient-boosted trees; Section V-C) that maps these 13 numbers to $\hat{s}_0 \in [0, 1]$, a prediction of the edit similarity the zero-shot completion will achieve — in effect, an instance-level confidence score distilled from token-level signals.

Stage 2: CARD gate. If $\hat{s}_0 < T_{\text{RAG}}$, the model is classified as uncertain. We retrieve cross-file context with BM25 over sliding-window chunks (20-line windows, stride 10, top-k) following the RepoCoder convention [3], regenerate, and return the retrieved-context prediction. This same retrieval procedure is used on all four benchmarks; only the corpus it ranks differs per benchmark (the benchmark-shipped cross-file fragments for CrossCodeEval and CrossCodeLongEval, live windows over the repository's other Python files for RepoEval; Section IV-C). Two properties hold everywhere: the query is the last 20 lines of X_{left} — the ground truth is never part of any query — and the corpus is strictly cross-file, with the completion's own file excluded, so the retrieved context can never contain the reference solution.

Stage 3: static-analysis gate. If CARD said skip, we run pyflakes over $X_{\text{left}} + \hat{y}_0 + X_{\text{right}}$ and flag any identifier used in $\hat{y}_0$ that is not defined anywhere in the current file. If the check fires, we retrieve and regenerate exactly as in Stage 2. Otherwise we yield $\hat{y}_0$ as the final prediction.

C. Data

The primary evaluation surface is the Python split of CrossCodeEval [4], which contains roughly 2,700 line-completion instances drawn from approximately 420 GitHub repositories. The benchmark authors selected its source repositories to limit training-data overlap with the code language models available at the time of its release. Each instance is one JSONL record providing the in-file context (X_{left} and X_{right}), the held-out completion (groundtruth), and a cross-file context block that is the only view of R available to the BM25 retriever. The static-analysis gate, by contrast, uses only the in-file context X (Section IV-D).

Each CrossCodeEval cross-file block is a set of 5 pre-retrieved passages selected by BM25 from the instance's upstream repository R. Each chunk is a contiguous 10-line window of a source file. The benchmark authors pre-tile every file in the upstream project into non-overlapping windows of that size and, for each instance, score those windows against the last 10 lines of X_{left}, returning the top matches. This chunk list is consumed only by the BM25 retriever, when retrieval fires. The static-analysis gate does not use these chunks as it resolves identifiers against the in-file context X alone.

As a secondary surface we evaluate on the function split of RepoEval [3], which contains 455 multi-line function-body completion instances drawn from the 8 Python repositories. We analyze function-level completion because multi-line generation is where structural hallucinations have room to appear. Rather than shipping a pre-retrieved chunk list per instance, as CrossCodeEval does, RepoEval ships the upstream repositories on disk in full. Thus, BM25 is run at inference time with 20-line sliding windows at stride 10 computed live from every other Python file of the target repository (all of the repository's files except the one being completed), with the top ten matches returned per query. The repository's Python files are loaded only to drive this live BM25 retrieval. Similarly to CrossCodeEval, the static-analysis gate consults only the in-file context X, not repository-wide information. We adopt these settings unchanged so our enhancement of the CARD baseline remains directly comparable to its published numbers [2].

To stress-test the models on longer, more hallucination-prone targets, we additionally evaluate on CrossCodeLongEval [5], the long-form benchmark introduced with Repoformer, in both its function (multi-line body) and chunk (fixed 1–6 line block) Python tasks (5,000 instances each). CrossCodeLongEval ships, per instance, a pre-built left/right in-file context and a retrieved cross-file block, which we use exactly as for CrossCodeEval. All four evaluation surfaces and their statistics are presented together in Section IV-E.

Summary: what each benchmark supplies. All three benchmarks provide, per instance, the in-file context X and the gold completion; they differ in the completion granularity and in how the cross-file corpus reaches us. CrossCodeEval (single-line) and CrossCodeLongEval (chunk and function) ship a pre-retrieved set of cross-file fragments per instance — selected by the benchmark authors with a query built from the context above the hole, never from the ground truth (CrossCodeLongEval's release labels its queries "query-without-gt") — and our BM25 ranks within that shipped set at inference. RepoEval ships whole repositories, so our BM25 builds its windows live over the repository's files. In every case retrieval is within-repository and cross-file: chunks come only from other files of the same project (never from another repository, and never from the file being completed), and every query is the last 20 lines of X_{left}, so no retrieval decision ever sees the ground truth.

D. Static-Analysis Gate

The static gate asks one question of the zero-shot completion $\hat{y}_0$: does it use any identifier that is not defined anywhere in the file being edited? A confident call to a helper function that exists nowhere is exactly the structural hallucination the confidence gate cannot see. We answer the question with the pyflakes static checker [1] rather than a custom analyzer:

(1) Reconstruct the file. We splice the completion back into its context, forming $X_{\text{left}} + \hat{y}_0 + X_{\text{right}}$, and

[1] github.com/PyCQA/pyflakes

TABLE I
Evaluation surfaces. "GT lines" is the ground-truth completion length in lines; "single" is the fraction of single-line targets. Scoring mode is the span the prediction is truncated to before computing accuracy. (Section V-B).

Benchmark	Task	#Inst.	#Repos	GT lines			Scoring
				med	p99	max	
CrossCodeEval [4]	line	2665	471	1	1	1	first line
RepoEval [3]	function	455	8	8	27	29	body
CCLong [5]	function	5000	944	9	30	31	body
CCLong [5]	chunk	5000	1460	1	4	6	line-count

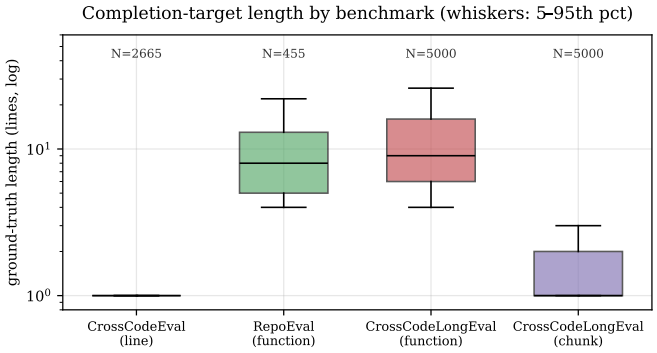

Fig. 1. Completion-target length by benchmark (log scale, box = IQR, whiskers 5–95th percentile). The four surfaces form a granularity spectrum: single-token/ single-line (CrossCodeEval), short fixed blocks (CCLong-chunk, 66% single-line), and multi-line function bodies (RepoEval, CCLong-function, median 8–9 lines).

record the line span that $\hat{y}_0$ occupies. (2) Run pyflakes. pyflakes resolves every name against the file's own bindings such as imports, function/class definitions, assignments, parameters, comprehension variables, with/except targets, the walrus operator, and so on — using a complete lexical binding model, ignores names that appear inside strings or comments, and emits an undefined name report (code F821) for each use of a name that nothing in the file binds. (3) Restrict and decide. We keep only the undefined-name reports that fall on $\hat{y}_0$'s own lines (discarding any caused by the surrounding context) and fire the gate — triggering retrieval — if at least one remains.

The check is purely in-file: it consults the imports and definitions already present in X, needs no repository-wide information, and runs in milliseconds. Two properties matter for soundness. First, because pyflakes uses a full binding model it does not false-positive on legitimate bindings (e.g. a match capture or a with … as alias) that a positional heuristic would miss. Second, if the reconstructed file fails to parse, the gate abstains (it does not fire), so a malformed completion can never result in a spurious trigger. This pyflakes gate is what produces every cascade ($C4$) decision and every hallucination number we report (Section VI).

E. Datasets

Our proposed approach is training-free: the static gate parses code, the CARD estimator is the only learned component and is calibrated once per model, not per dataset. Hence, our data requirement is entirely on the evaluation side. We therefore evaluate on four Python datasets drawn from three established benchmarks; their statistics are summarised in Table I and Fig. 1.

Single-line: CrossCodeEval (line). Every target is exactly one line (median 38 characters); the model emits a short, mostly local completion. We include it as the easy end of the spectrum and as the setting used by our CARD baseline [2]: with so little generated code, there is little room for a structural hallucination, so this surface is a sanity check on which we expect the static gate to fire rarely and improve marginally.

Multi-line functions: RepoEval (function) and CCLong (function). Here the hole is an entire function body: a median of 8–9 lines and up to 31. This is the regime we find most interesting for two reasons. First, generating many lines gives the model many opportunities to reference helper functions, classes, or attributes that do not exist, so we expect structural hallucinations to be most frequent here, being the dataset on which we want to "push the models and see where they hallucinate." Second, function completion is what lets us introduce truncation: a code LM asked for one function body keeps generating into the next definition, so the raw output must be cut at the end of the body before it is scored. We truncate at the dedent boundary of the generated body (the Python analogue of CARD's bracket-matching), which materially changes the measured accuracy (Section VI) and, as we show, must be handled carefully so that it does not interact with the static check.

Fixed blocks: CCLong (chunk). The chunk task completes a fixed window of 1–6 lines (66% are single-line, median 1, max 6). It sits between the single-line and function regimes: short enough that targets are often a single statement, but multi-line often enough (34%) that the prediction must be truncated to the gold's line count — matching Repoformer's chunk metric — rather than to a function body. Concretely: a chunk is an arbitrary span of lines, not a syntactic unit, so there is no natural boundary (like a function's dedent) at which to stop scoring. The model, however, keeps writing past the span. Following Repoformer, we therefore count the gold's non-blank lines — say three — keep only the prediction's first three non-blank lines, and score those against the gold; everything the model wrote beyond the target span is ignored by the accuracy metrics (but, as always, still visible to the hallucination metrics, which read the complete output). We use it to test whether our findings on functions degrade gracefully to shorter, less structurally rich targets.

Why this selection. The benchmarks were curated to reduce training-data contamination: CrossCodeEval samples permissively licensed repositories created after

common pre-training cutoffs; CCLong is built from 1,500 held-out repositories, which protects the credibility of the no-retrieve baseline. More importantly for our study, the four surfaces deliberately bracket the hypothesised effect: the smaller, shorter targets (CrossCodeEval line, CCLong chunk) are where we expect few hallucinations and therefore act as controls, while the longer function-body targets (RepoEval, CCLong function) are where we expect the static gate to matter most. Confirming that the gate is both inactive on the controls and strongly beneficial on the long targets is exactly the evidence our claim needs.

V. Experiment Setup

A. Models and Configurations

We evaluate three open-weight code LMs spanning an order of magnitude in size: Qwen2.5-Coder-0.5B and Qwen2.5-Coder-1.5B [6] and CodeLlama-7B [7]. All are used zero-shot (no fine-tuning of the generator). The only learned component is CARD's critique estimator. We compare four configurations, all sharing the same generator and retriever so differences isolate the gating policy: C1 no-retrieve (in-file context only), C2 always-retrieve, C3 CARD (retrieve iff $\hat{s}_0 < T_{\text{RAG}}$), and C4 the cascade (C3, plus retrieve when the static-analysis gate fires on a skipped instance). Rather than reporting C3/C4 at one threshold, we sweep $T_{\text{RAG}} \in \{0.05, \ldots, 0.95\}$ to trace the entire cost–accuracy operating curve.

B. Evaluation Setting and Metrics

Setting. All generation is greedy (temperature 0), so results are deterministic and re-runs are exact. Test/train overlap is mitigated upstream by the benchmarks' curation (Section IV-E); we add no fine-tuning that could leak test data. Because a code LM asked for a multi-line body keeps generating past it, the prediction is truncated before accuracy is scored: to the first line for CrossCodeEval, to the generated function body (dedent boundary) for the function tasks, and to the ground-truth line count for chunks (matching Repoformer). We report every accuracy metric both on this truncated span (the headline) and on the raw, untruncated output, so the effect of truncation is visible rather than hidden.

Accuracy metrics. Exact Match (EM) is the fraction of predictions equal to the ground truth after truncation. Edit Similarity (ES) is $1 - \text{Lev}(\hat{y}, y)/\max(|\hat{y}|, |y|) \in [0, 1]$, the standard character-level metric used by CARD and CrossCodeEval [2], [4]. Identifier-F1 (idF1) is the set-F1 over identifier tokens, the identifier-matching metric of CrossCodeEval [4].

Hallucination metrics. Our target failure mode is the invented identifier: a name the model emits that is both (A4) absent from the ground truth and (B2) unresolvable by static analysis. The labels come from the metric catalogue we drafted for this project: family A collects reference-based conditions (computed against the gold) and family B collects static-analysis conditions (computed by a checker), with A4 = "emits an identifier not used by the gold" and B2 = "the checker reports the identifier as undefined". Neither condition is a good hallucination signal alone: A4 by itself over-counts, because a valid alternative name that the gold simply did not use is not a hallucination; B2 by itself is circular with our gate, because the cascade triggers on the same undefined-name analysis. Their conjunction isolates exactly the names that are both un-referenced and unresolvable — "made up" in the everyday sense. We report h_{A4B2}, the fraction of predictions containing at least one identifier satisfying both conditions (B2 via pyflakes' F821 undefined-name analysis on the reconstructed file), as the strict "made-up name" signal, and the looser h_{A4} (absent-from-gold only) as an upper bound. Crucially, the hallucination metrics are always computed on the model's complete (untruncated) output: truncating to an oracle length can delete a definition the rest of the file uses — thereby manufacturing an undefined-name flag that the model never produced — and the gold length is unavailable at inference. Computing the structural signal on the complete generation both avoids this artefact and matches what a deployed gate would see. The same principle governs the cascade's static-gate trigger.

Efficiency metrics. We report the retrieval rate (% of instances for which retrieval fired)— the fraction of instances that pay for a retrieval-augmented second pass, i.e. the quantity a gating policy directly controls and a hardware-independent proxy for its cost — and per-instance latency (ms). For the adaptive configs latency is the cost model of CARD/Repoformer: a zero-shot pass for every instance plus a second retrieval-augmented pass only when the gate fires.

C. Configuration and Implementation Details

Generation. We use fill-in-the-middle (FIM) prompting [8] with each model's native FIM sentinels, plus a set of stop strings that halt generation at the natural boundary (preventing the <|fim_pad|> degeneration that otherwise floods short completions and corrupts both the text and CARD's per-token features). The generation budget is set per task to the 99th percentile of gold length in tokens — 50 (CrossCodeEval), 280 (RepoEval-function), 400 (CCLong-function), and 80 (CCLong-chunk) — so the model can express the full target without unbounded over-generation.

Adaptive gate. CARD's critique estimator predicts the zero-shot output's edit similarity from a 13-dimensional aggregation of the generator's per-token probabilities and entropies [2]; we implement it as a LightGBM regressor [9]. Because those features are extracted from the generator's own logits, the estimator is model-specific: we calibrate three separate estimators, one per generator (Qwen2.5-Coder-0.5B, Qwen2.5-Coder-1.5B, and CodeLlama-7B), each on self-supervised samples from The Stack (dedu-

plicated) [10] following the CARD procedure [2]. Each estimator is a property of its generator, not of any benchmark, so it is calibrated once and reused unchanged across all four datasets. LightGBM itself is a gradient-boosted decision-tree (GBDT) learner: the regressor is an ensemble of small regression trees built sequentially, each new tree fitting the residual error of the ensemble so far, with histogram-based split finding for speed. We chose it for the same reasons CARD did: on a 13-feature tabular input it is accurate with little tuning, trains in seconds–minutes on CPU, and predicts in well under a millisecond — negligible next to a generation pass. The static-analysis gate is realised with pyflakes over the reconstructed file (Section IV-D). Retrieval is Okapi BM25 [11] over 20-line/stride-10 windows, returning the top-10 chunks, following the RepoCoder convention [3]. One retrieval subtlety mattered enough to flag: the loaders keep a copy of the current file inside each instance's file map (other components need it), and an early version of our harness let BM25 index it. Because the query is the 20 lines directly above the hole, the window overlapping the hole — which contains the reference solution — then ranked in the retrieved top-10 for 92–100% of instances, silently inflating every retrieval-augmented score. The final harness excludes the completion's own file from the corpus by construction; we verified post-fix that no retrieved chunk comes from the target file on any benchmark, and all numbers reported in this paper are produced with the corrected, leak-free corpus.

Infrastructure and scale. All inference runs on a single NVIDIA A100-80GB GPU (Google Colab) using vLLM [12] with PagedAttention and continuous batching; the chip is fixed across models so the reported latencies are comparable within a benchmark. We batch the C1/C2 generation passes and share a generation cache across configurations. The full T_{RAG} sweep is then computed post hoc by replaying CARD and the cascade over the frozen generations, so no instance is generated twice. In total the study covers 13,120 instances × two generation passes × three models, plus the 19-point threshold sweep.

VI. Results

We answer the four research questions in turn. Table II gives the two fixed-policy endpoints (C1 never retrieve, C2 always retrieve) for every benchmark and model; Table III isolates the cascade's contribution against CARD at a low retrieval budget; Table IV reports the truncated-vs-raw accuracy. Figures 3–6 visualise the trade-offs over the full threshold sweep.

A. RQ1: Retrieval is beneficial on average, but far from uniformly

The C1→C2 columns of Table II show that adding cross-file context helps on every surface and model, but modestly: edit similarity rises by +0.03 to +0.08 and exact match by a few points (e.g. RepoEval 1.5B: EM 0.06 → 0.13). The no-retrieve baselines are "correctly low" — the models are knowledge-limited, not broken, which we confirmed by manual inspection (C1 outputs are fluent but reference wrong, locally plausible names) — yet feeding the missing context back as commented prompt text recovers only part of that headroom for these base code LMs. Figure 2 decomposes the effect per instance: always-retrieve helps only 20–41% of instances, leaves 35–66% essentially unchanged, and actively hurts 14–24%. This matches the selective picture reported by Repoformer [5], where a large fraction of retrievals do not improve the prediction: most of the always-retrieve budget is spent on instances it cannot help, and a sizeable slice is harmed by it — precisely what an adaptive policy should reclaim. Retrieval is therefore worth gating, motivating RQ2–RQ4.

Answer (RQ1). Cross-file retrieval helps on average on every surface and model, but only a minority of instances (20–41%) actually benefit; most are unchanged and up to a quarter are degraded, so an always-retrieve policy wastes most of its budget — selective retrieval is justified.

B. RQ2: CARD trades cost for accuracy efficiently, but leaves a structural gap

Figure 3 traces accuracy against retrieval budget as T_{RAG} is swept. The cascade (and CARD, which nearly coincides on ES) recovers the — modest — C1→C2 accuracy gain well before reaching a 100% retrieval rate, so a confidence gate buys retrieval's benefit at a fraction of its cost: the efficiency claim of CARD and Repoformer [2], [5] reproduces on our benchmarks and models, even though the absolute ceiling is low in our base-model FIM setup. Figure 4 shows the same trade-off in wall-clock terms: edit similarity and per-instance latency rise with T_{RAG} together. However, the confidence gate is blind to one thing: the h_{A4B2} columns of Table II show that always-retrieving increases invented identifiers in every one of the twelve settings (e.g. RepoEval 7B 0.018 → 0.031; CCLong-function 7B 0.042 → 0.067; CCLong-function 1.5B 0.034 → 0.061) — retrieved context invites the model to write more, and more adventurous, code, emitting more unresolvable names. A policy that only optimizes confidence/ES therefore leaves a structural failure mode untouched — and paying for more retrieval makes it worse, not better. This is the gap our static gate targets.

Answer (RQ2). A small retrieval budget captures the (modest) accuracy gain of always-retrieving — the confidence gate trades cost for accuracy efficiently — but it is blind to structural validity: invented identifiers increase under retrieval in every setting, leaving exactly the failure mode a structural signal can address.

C. RQ3: The static gate suppresses the hallucinations CARD misses

Table III compares CARD (C3) and the cascade (C4) at a low retrieval budget, obtained with the strictest

TABLE II

Endpoints: C1 (never retrieve) vs. C2 (always retrieve), headline (truncated) scoring. ES/EM/idF1 higher is better; h_{A4B2} (invented-identifier rate) lower is better; ms is per-instance latency under batched inference. Note that C2 lifts accuracy only modestly, while h_{A4B2} rises under C2 in every setting: retrieval does not fix invented identifiers — it amplifies them.

		C1 no-retrieve					C2 always-retrieve				
Dataset	M	ES	EM	idF1	h_{A4B2}	ms	ES	EM	idF1	h_{A4B2}	ms
CCE (line)	0.5B	0.556	0.205	0.518	0.0428	199	0.581	0.232	0.551	0.0522	33
	1.5B	0.631	0.294	0.603	0.0210	200	0.665	0.328	0.642	0.0263	53
	7B	0.652	0.315	0.626	0.0169	502	0.692	0.368	0.675	0.0266	240
RepoEval (func)	0.5B	0.357	0.046	0.483	0.0835	1280	0.397	0.055	0.524	0.1275	194
	1.5B	0.431	0.055	0.585	0.0527	1106	0.513	0.132	0.660	0.0703	282
	7B	0.444	0.084	0.558	0.0176	3607	0.498	0.123	0.617	0.0308	1270
CCLE (func)	0.5B	0.397	0.039	0.520	0.0660	153	0.433	0.069	0.555	0.0976	196
	1.5B	0.468	0.080	0.605	0.0342	170	0.510	0.114	0.640	0.0606	249
	7B	0.489	0.085	0.616	0.0418	904	0.529	0.120	0.651	0.0670	1513
CCLE (chunk)	0.5B	0.605	0.265	0.580	0.0208	58	0.643	0.314	0.614	0.0330	93
	1.5B	0.707	0.394	0.690	0.0164	102	0.736	0.454	0.717	0.0222	169
	7B	0.711	0.419	0.691	0.0206	422	0.744	0.482	0.727	0.0278	699

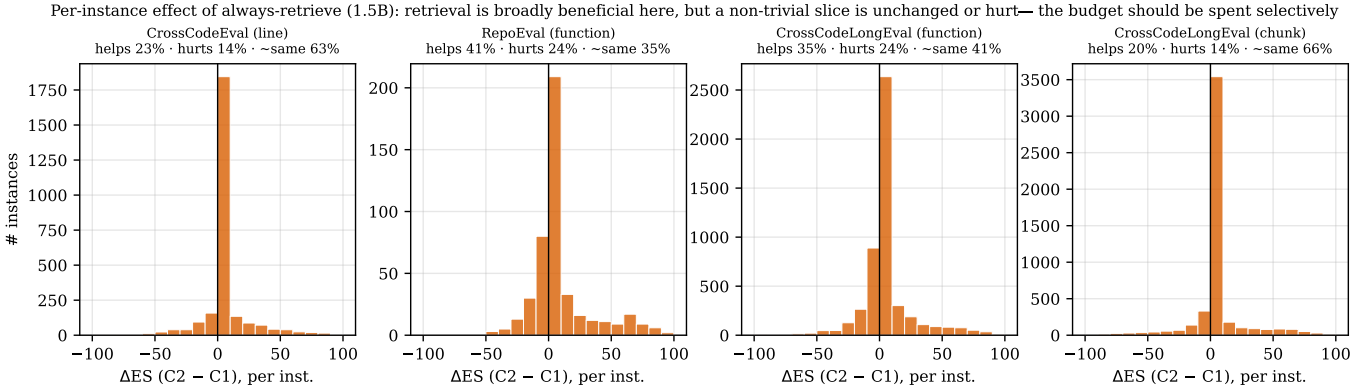

Fig. 2. Per-instance effect of always-retrieve (1.5B): ΔES = ES(C2)−ES(C1). Only a minority of instances benefit from retrieval; most are unchanged and a sizeable fraction is degraded — the budget should be spent selectively rather than uniformly (RQ1).

threshold ($T_{\text{RAG}} = 0.05$, where CARD is most conservative and retrieves almost nothing). Threshold and budget are two views of the same knob: the threshold is what we set, and the realised retrieval rate — the budget actually spent — is what it produces. Adding the static-analysis gate retrieves an extra 2–9% of instances — precisely those whose confident zero-shot output references an unresolvable name — and cuts the invented-identifier rate by 1.9× to 8× (e.g. RepoEval-7B 0.018 → 0.002, an 8× drop). The effect is consistent across all twelve (benchmark × model) cells. Figure 5 places this on the budget axis: throughout the low-budget regime where a selective policy actually operates, the cascade (solid) sits below CARD (dashed) at a matched retrieval rate. As the budget grows the two converge — and can cross — because both approach always-retrieve, whose own hallucination rate is higher than no-retrieve's (RQ2): once a policy retrieves almost everywhere, what dominates is no longer which instances are selected but the fact that retrieval itself inflates invented identifiers. Figure 6 summarises the headline reduction as paired bars. Because the trigger and the h_{A4B2} metric both rest on undefined-name analysis, part of this drop is structural by construction; we therefore also verified that the looser, fully independent h_{A4} (absent-from-gold) moves in the same direction, and that edit similarity does not fall (Section VI-D).

Answer (RQ3). Yes: triggering on unresolvable identifiers retrieves precisely the instances the confidence gate wrongly keeps, cutting the invented-identifier rate severalfold at a few percent of extra retrieval, consistently across all twelve settings (statistical significance reported in Section VII-C).

D. RQ4: The gate is free and scales as hypothesised

No accuracy regression. The cascade obeys an asymmetry guarantee by construction — it only adds retrieval to CARD's skip decisions — and the data confirm it empirically: across all 228 (threshold × setting) points, C4 retrieval rate ≥ C3 and C4 $h_{A4B2} \leq$ C3 everywhere, and C4 ES ≥ C3 in 226 of 228 points (the two exceptions are mean-ES dips of 10^{-4}, below reporting precision)

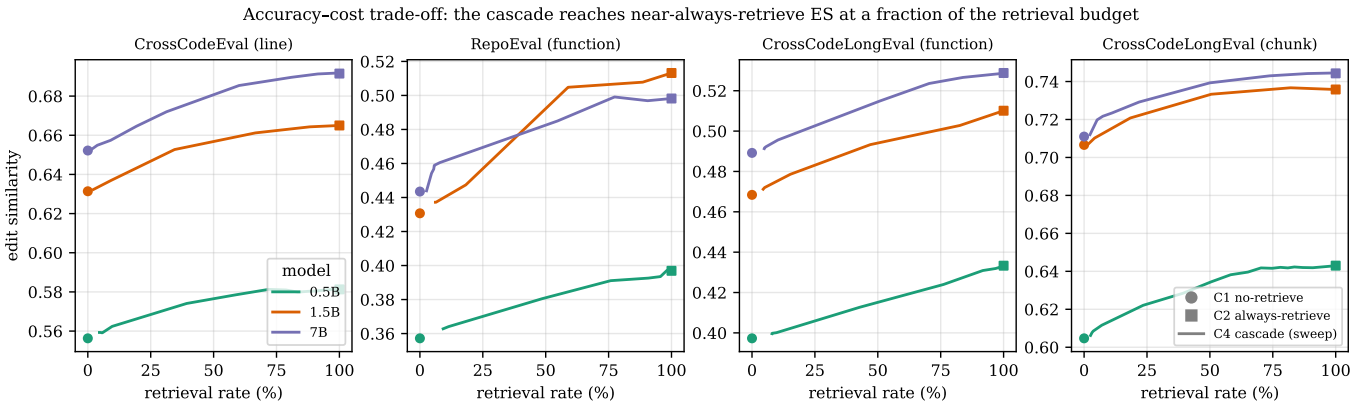

Fig. 3. Accuracy–cost trade-off (RQ2). Edit similarity vs. retrieval rate as T_{RAG} is swept (C4 cascade); • = C1, ■ = C2. The C1→C2 gain is modest in our setup, and the selective frontier captures it well below a 100% retrieval budget, for all three models and all four surfaces.

TABLE III

RQ3 — the cascade's contribution at a low budget/threshold ($T_{\mathrm{RAG}} = 0.05$). For a few percent of extra retrieval (r%), the static gate cuts the invented-identifier rate h_{A4B2} by the factor in the last column, while edit similarity is non-decreasing. "→0" denotes a drop to zero.

		C3 CARD			C4 cascade (ours)			
Dataset	M	r%	ES	h_{A4B2}	r%	ES	h_{A4B2}	↓
CCE (line)	0.5B	0	0.557	0.0428	5	0.559	0.0154	2.8×
	1.5B	0	0.631	0.0210	2	0.632	0.0071	3.0×
	7B	0	0.652	0.0169	2	0.653	0.0090	1.9×
RepoEval (func)	0.5B	0	0.357	0.0835	9	0.363	0.0308	2.7×
	1.5B	0	0.431	0.0527	6	0.437	0.0110	4.8×
	7B	0	0.444	0.0176	3	0.444	0.0022	8.0×
CCLE (func)	0.5B	0	0.397	0.0660	8	0.400	0.0260	2.5×
	1.5B	0	0.468	0.0342	4	0.471	0.0144	2.4×
	7B	0	0.489	0.0418	5	0.491	0.0172	2.4×
CCLE (chunk)	0.5B	0	0.605	0.0208	3	0.606	0.0080	2.6×
	1.5B	0	0.707	0.0164	2	0.708	0.0064	2.6×
	7B	0	0.711	0.0206	2	0.712	0.0094	2.2×

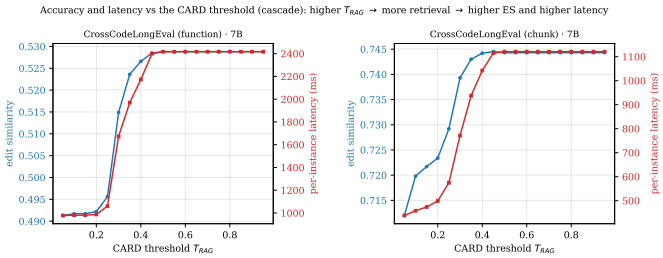

Fig. 4. Accuracy and latency vs. the CARD threshold (7B cascade): both rise with T_{RAG} and plateau, so the threshold is a single knob trading per-instance latency for accuracy (RQ2).

(Table III shows ES ticking up slightly, e.g. CCLong-function 0.5B 0.397 → 0.400). The hallucination reduction is thus obtained at no cost to accuracy and only single-digit-percent extra retrieval (hence near-C1 latency).

Scaling. Accuracy rises monotonically with model size on every surface (Table II). Invented-identifier rates fall with size on the cleaner benchmarks (CCE C1 h_{A4B2}: 0.043/0.021/0.017; RepoEval: 0.084/0.053/0.018 for 0.5B/1.5B/7B) but are noisier on the longer CCLong targets (7B-function 0.042 $\gtrsim$ 1.5B 0.034) — longer, harder contexts make even the 7B invent names, which is exactly where the gate keeps paying off. The contribution is largest in absolute terms where hallucination is largest (smaller models, function tasks) and is near-inert on the single-line control (Fig. 6), as intended.

Answer (RQ4). The gate is effectively free: it adds only single-digit-percent retrieval and does not sacrifice edit similarity at any operating point, and its benefit concentrates exactly where hallucination lives — smaller models and function-length targets — while staying near-inert on the single-line control.

Truncation matters. Table IV shows the headline (truncated) ES is far above the raw ES on every multi-line surface (e.g. 7B CCLong-function C2 0.529 truncated vs. 0.333 raw): without body/line truncation, over-generation past the target span dominates the score. This both validates the truncation step and underlines why the structural signal must be computed on the complete

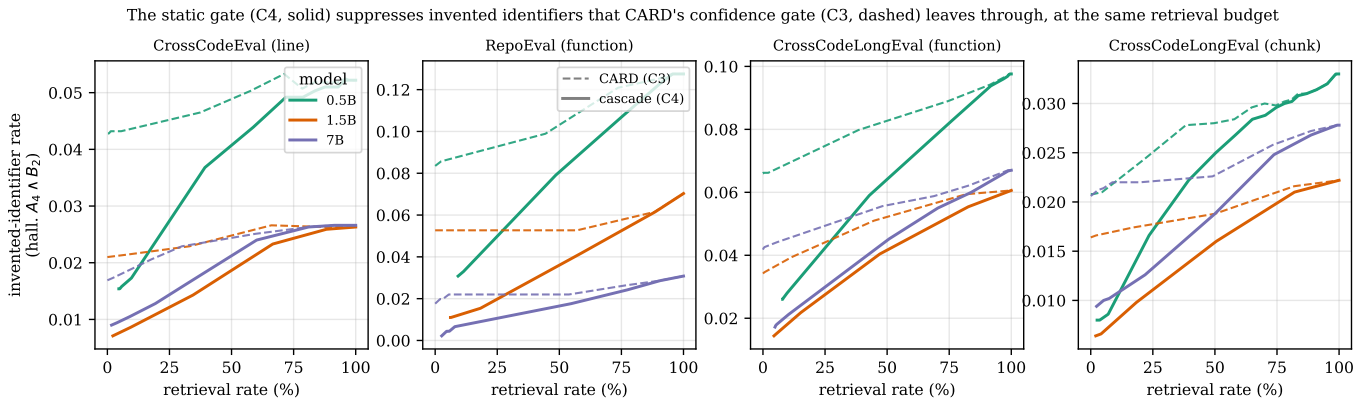

Fig. 5. Invented-identifier rate vs. retrieval budget (RQ3). In the low-budget regime where a selective policy operates, the cascade (C4, solid) lies below CARD (C3, dashed) at a matched budget for every model; at high budgets both approach always-retrieve — whose hallucination rate exceeds no-retrieve's — so the curves rise and converge (RQ2).

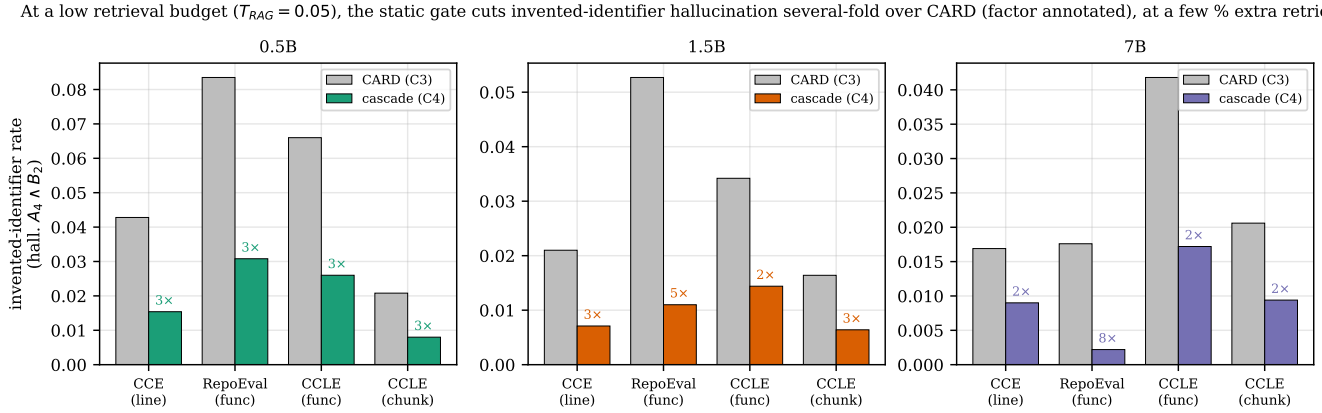

Fig. 6. Headline reduction (RQ3): invented-identifier rate for CARD vs. the cascade at $T_{\mathrm{RAG}} = 0.05$; the annotated factor is the reduction. The gate is near-inert on the easy single-line control yet strongly beneficial on the function tasks, exactly as hypothesised.

TABLE IV
RQ4 — truncated (headline) vs. raw edit similarity. Over-generation past the target span heavily deflates the raw score on every multi-line surface, justifying span truncation for accuracy (and on-complete scoring for hallucination).

		C1 ES		C2 ES	
Dataset	M	trunc.	raw	trunc.	raw
RepoEval (func)	0.5B	0.357	0.265	0.397	0.283
	1.5B	0.431	0.374	0.513	0.403
	7B	0.444	0.330	0.498	0.332
CCLE (func)	0.5B	0.397	0.287	0.433	0.287
	1.5B	0.468	0.389	0.510	0.390
	7B	0.489	0.356	0.529	0.333
CCLE (chunk)	0.5B	0.605	0.478	0.643	0.429
	1.5B	0.707	0.575	0.736	0.546
	7B	0.711	0.468	0.744	0.440

output rather than the truncated one (Section V-B).

VII. Discussion

Confidence is not structure. The adaptive-retrieval line of work gates on the model's own uncertainty: FLARE [13] re-retrieves when the next sentence contains low-probability tokens, and CARD [2] trains a critique model to predict the zero-shot output's edit similarity from logit statistics. Our results make concrete a limitation implicit in both: token-level confidence is necessary but not sufficient. A model can be perfectly confident in a call to a helper that does not exist — the tokens are individually likely because the shape is right — and exactly these cases slip through the confidence gate. The static check is cheap, training-free, and complementary: it reads the same zero-shot output CARD already produced and asks an orthogonal question ("does this resolve?") rather than a confidence question ("is this likely?").

Retrieval is not a hallucination cure. Repoformer [5] showed that always-retrieving is wasteful for accuracy — up to 80% of retrievals do not help, a proportion we reproduce (Fig. 2). We add a sharper, complementary

observation: in our setup always-retrieving consistently worsened structural hallucination — h_{A4B2} rose under C2 in all twelve settings (Table II) — because more context invites more (and longer) generation, and thus more opportunities to invent names. This reframes selective retrieval: the goal is not only to skip unhelpful retrievals but to spend the budget on the structurally broken outputs, which is exactly what the static gate does. It also explains why the cascade's advantage is concentrated at low budgets (Fig. 5): once a policy retrieves almost everywhere, selectivity — and hence the gate's targeting — has no room to act.

Why functions. The contribution tracks generation length: near-inert on single-line CrossCodeEval, strongest on function bodies (Fig. 6). This is intuitive — a one-line completion rarely has space to call a non-existent function — and it is why benchmarks like CCLong [5], which deliberately lengthen the target, are the right testbed for structural hallucination. It also surfaces a scoring pitfall the community should heed: a model asked for one function over-generates into the next, so the two measurements need different views of the output — accuracy must be scored on the span-truncated prediction (Table IV), while the structural signal must see the complete generation. Truncating the output before static verification conflates these two requirements, falsely manufacturing hallucinations that the model never actually made.

A. Implications

For practitioners, the static gate is a near-zero-cost add-on to any confidence-gated RAG completion system: it needs no training, runs in milliseconds on the already-generated output, and removes the bulk of invented-identifier hallucinations at a few percent extra retrieval and no accuracy loss — a favourable trade whenever a confidently wrong completion costs the user more than a brief retrieval delay. For the research community, our results argue that (i) selective-retrieval policies should consider structural signals, not only confidence; (ii) repository-level completion should be evaluated with an explicit structural-hallucination metric, not edit similarity alone, since retrieval can improve ES while leaving (or worsening) hallucination; and (iii) such metrics must be computed on the model's complete output, decoupled from the accuracy-side truncation. The full per-instance results and the harness that sweeps any gate over the budget axis are released to support this.

B. Future Work

Several directions follow directly. First, bidirectional gating: the static gate only adds retrieval to CARD's skip decisions; a completion that resolves cleanly is only weak evidence of correctness, but combined with further training-free structural signals it might also be used to suppress unnecessary retrievals, tightening the cost side of the trade-off. Second, repair instead of retrieve: the static analyzer already localises the offending identifier, so a cheaper action than full retrieval may be to constrain or correct that token. Third, better function-level critique labels: Repoformer notes that ES-based self-evaluation is miscalibrated for function completion, and our CARD estimator inherits this; a structural or execution-based label could sharpen the confidence gate so the cascade starts from a stronger base. Fourth, broader coverage: more model families and sizes, and languages beyond Python (the gate is language-agnostic in principle, needing only a parser and a scope resolver). Finally, repository-adaptive thresholds, since duplication and modularity vary across projects [3].

C. Threats to the Validity

Construct. Edit similarity and exact match measure surface overlap, not functional correctness; we do not run tests (pass@k), so a "correct" completion may still be semantically wrong, and vice versa. Our hallucination metric is a static proxy: pyflakes cannot resolve dynamically-created or star-imported names, which could flag a valid identifier as undefined; we mitigate this with pyflakes' complete binding model (which ignores names in strings and comments and recognises every standard binding form) and by computing the signal on the reconstructed file, which contains the in-file imports and definitions. There is a partial circularity: the cascade's trigger and the h_{A4B2} metric both rest on undefined-name analysis, so some of the measured reduction is structural by construction; we address this by also reporting the independent absent-from-gold rate h_{A4} (which moves the same way) and by showing edit similarity never decreases.

Internal. We re-implemented the CARD baseline rather than using original code, and reused the 1.5B/7B critique estimators across datasets; a calibration difference would shift C3 (and thus the C3→C4 gap) but not the asymmetry guarantee, which is structural. A retrieval-corpus bug we caught during this round (the completion's own file was indexable by BM25, letting the reference solution reach the prompt; Section V-C) was fixed and the retrieval-augmented generations underlying every reported number (the C2 pass, which the threshold sweep replays for C3/C4) were regenerated with the leak-free corpus; we verified post-fix that no retrieved chunk originates from the target file on any benchmark. Generation is greedy and deterministic, so results are reproducible but reflect a single decode. Latencies are measured under vLLM continuous batching and are therefore throughput-amortised; we use them only for within-benchmark, same-hardware comparison and rely on retrieval rate as the hardware-independent cost proxy. We complement the point estimates with a paired exact McNemar test on the per-instance invented-identifier flags of CARD vs. the cascade at $T_{\mathrm{RAG}} = 0.05$: the reduction is statistically significant in all twelve settings ($p < 10^{-5}$ in eleven of them; $p = 0.016$ on RepoEval-7B, where only eight flagged

instances remain to begin with), and in nine of the twelve the cascade introduces zero newly-flagged instances — it only removes them.

External. Findings cover three code LMs (0.5B–7B), Python only, and four benchmarks; larger models hallucinate less (Table II), so the absolute benefit may shrink at frontier scale even as the asymmetry guarantee persists. The CCLong-chunk surface strips blank lines during its construction, which does not affect Python scoping but makes its raw text differ slightly from on-disk source. Results on other languages or proprietary models may differ.

VIII. Conclusion

We set out to close a structural blind spot in adaptive retrieval for repository-level code completion: confidence gates such as CARD skip retrieval on outputs that are fluent but reference identifiers that exist nowhere in the repository. Our remedy is a training-free static-analysis gate that runs only when the confidence gate decides to skip, parses the zero-shot completion, and triggers retrieval when it finds an out-of-scope identifier. Evaluated over four Python surfaces and three code LMs (13,120 instances), with the retrieval budget swept end to end, the cascade cuts the invented-identifier rate by 2–8× over CARD at a matched, single-digit-percent retrieval budget, does not sacrifice edit similarity at any operating point (the asymmetry analysis of Section VI-D), and addresses a failure mode that always-retrieving does not — and, in our experiments, consistently worsens. The effect is largest where it should be (smaller models, longer function targets) and near-inert on easy single-line completions. Beyond the specific gate, the study argues that selective retrieval should weigh structural evidence alongside confidence, and that repository-level completion should be measured with a hallucination metric computed on the model's complete output. We release the code, the full per-instance results, and the sweeping evaluation harness to make these comparisons reproducible.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[2] W. Zhang, T. Fu, T. Yuan, G. Zhang, D. Chen, and J. Wang, "A lightweight framework for adaptive retrieval in code completion with critique model," 2024.

[3] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 2471–2484.

[4] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in Advances in Neural Information Processing Systems 36 (NeurIPS 2023) Track on Datasets and Benchmarks, 2023.

[5] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma, "Repoformer: Selective retrieval for repository-level code completion," in Proceedings of the 41st International Conference on Machine Learning (ICML), ser. PMLR, vol. 235, 2024.

[6] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-Coder technical report," 2024.

[7] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin et al., "Code llama: Open foundation models for code," 2023.

[8] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen, "Efficient training of language models to fill in the middle," 2022.

[9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017.

[10] D. Kocetkov, R. Li, L. Ben Allal, J. Li, C. Mou, C. Muñoz Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf, D. Bahdanau, L. von Werra, and H. de Vries, "The stack: 3 TB of permissively licensed source code," 2022.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), 2023.

[13] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

IX. Appendix: Task Assignment Sheet

Instruction: After discussion among yourself within your group, list the main project-related tasks assigned to and done by each team member. Tasks can be: conducting a literature review, preparing a dataset, implementation of technique X, code review, fine-tuning a model, prompt design, setting up eval pipeline, evaluating the technique X, improving repo documentation, writing section X of the report, preparing the presentation (section methodology, section evaluation, etc.). The goal is to have a balanced task distribution among team members.